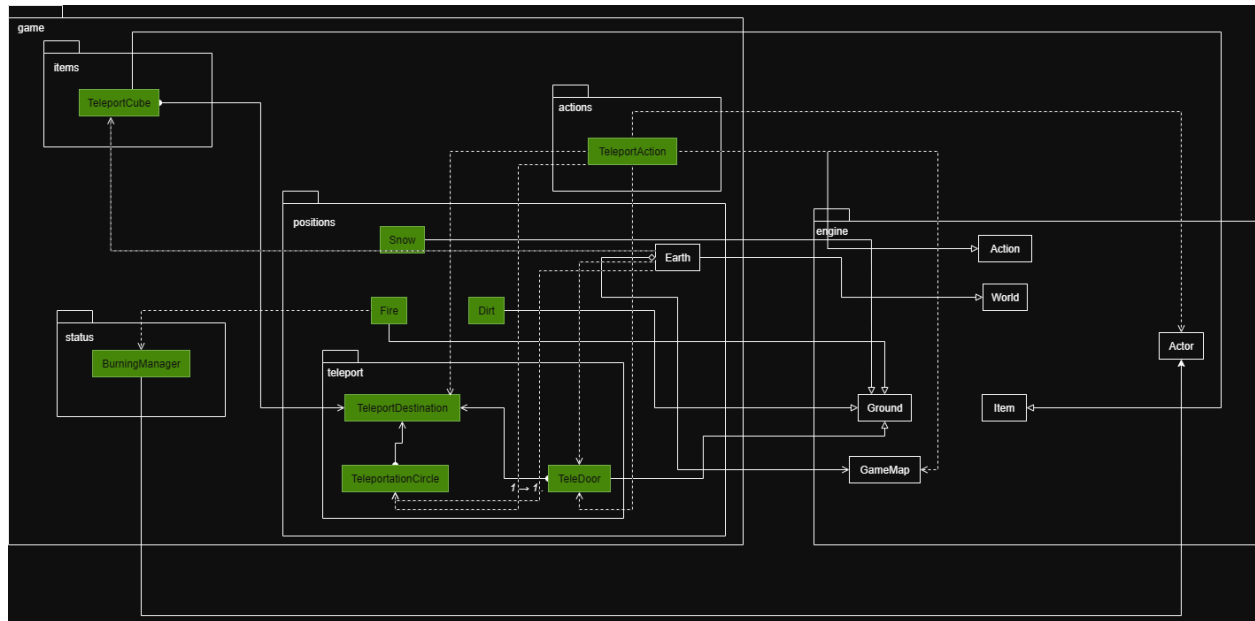


REQ1 Design Rationale

Introduction

REQ1 adds three teleportation sources—Tele-Door (#), Teleportation Circle (0), and a Teleport Cube plus environmental fire and stacking burn DoT. The solution must support intra-map and inter-map jumps, 50% cube malfunction, burn persistence and stacking, Fire→Dirt permanence, and integrate cleanly with the FIT2099 engine



Design A — Action-Centric Teleportation (Chosen)

A. How teleport is triggered

Design 1: In this design, the **Actor** itself checks the ground it is standing on every turn. If the ground is a teleport tile (e.g. Teleportation Circle or TeleDoor), the actor directly initiates the teleport logic. The responsibility for detecting and triggering teleportation is therefore actor-driven.

Design 2: In this design, the **Ground** object (e.g. TeleportationCircle) is responsible for triggering teleportation when an actor enters it. The ground calls or constructs a **TeleportAction**, which then performs the movement. This pushes the logic into the environment instead of the actor.

Design 1: Actor-driven polling	Design 2: Ground-driven hook
Pros: • Simple to implement inside Actor loop • No new callback needed	Pros: • Respects SRP/OCP • Lower coupling (actors don't know ground types) • Extensible to other ground-triggered behaviors
Cons: • Actor tightly coupled to specific grounds • Harder to extend (violates OCP) • High connascence of type/name	Cons: • Requires an onEnter hook to be implemented

- **Chosen design:** *Design 2 – Ground-driven hook*, as it reduces coupling and respects SOLID principles.
-

Design B — Per-Source Logic (Door/Circle/Cube own their teleports)

Design 1: Each **TeleDoor** or teleport tile directly references the destination tile or location. For example, one door might store a hard reference to another door or coordinate, making teleportation essentially a pairwise link.

Design 2: Teleport tiles use a **registry or identifier-based approach**. A `TeleportationCircle` or `TeleDoor` stores a destination ID, and the `TeleportAction` queries a central `TeleportDestination` registry to resolve the actual target location. This makes links indirect but flexible.

B. Destination resolution

Design 1: Pairwise doors	Design 2: Registry-based resolution
Pros: • Fast lookup • Easy to reason about	Pros: • Decoupled from specific doors • Supports retargeting destinations • Scales well across maps
Cons: • N^2 wiring between doors • Fragile if map changes • Strong coupling between door pairs	Cons: • Requires registry setup and destination IDs

- **Chosen design:** *Design 2 – Registry-based resolution*, as it supports extensibility and lowers coupling.

C. Performing the move

Design 1: The teleport tile (e.g. TeleDoor or TeleportationCircle) itself directly manipulates the game map to move the actor. The ground holds the responsibility for both triggering and performing the move.

Design 2: A dedicated **TeleportAction** encapsulates the move logic. The ground only triggers the action, but the action handles validation, moving the actor, and ensuring the engine rules are respected. Movement becomes an action rather than a side effect of the ground.

Design 1: Ground moves actor directly	Design 2: Encapsulate in TeleportAction
Pros: • Direct, fewer indirections	Pros: • SRP: actions handle movement • OCP: teleport rules can be extended • Consistent with engine's action execution model
Cons: • Grounds become "god objects" • Breaks encapsulation • Harder to test	Cons: • One extra class call per teleport

- **Chosen design:** *Design 2 – Action encapsulation*, for better separation of concerns.

D. Tile linkage (connascence)

Design 1: Teleport tiles are linked via **hardcoded positions or coordinates**. For example, a circle may assume that its matching door is at a fixed location. The relationship between source and destination is positional.

Design 2: Tiles are linked using **explicit identifiers**. A circle might store a symbolic destination key, and the registry resolves which door or location matches that key. This makes the link explicit and independent of physical layout.

Design 1: Positional/coordinate linkage	Design 2: Identifier-based linkage
Pros: • Quick prototype	Pros: • Lower connascence • Links are explicit and self-documenting • Flexible across maps
Cons: • Brittle if map layout changes • Hidden dependency on coordinates	Cons: • Requires assigning and registering IDs

Chosen design: *Design 2 – Identifier-based linkage*, as it minimises connascence.

E. Item trigger (TeleportCube)

Design 1: The teleport cube is handled as a **special case by the Actor**. When the actor uses the cube, it directly calls teleport logic in its own code path.

Design 2: The teleport cube works by issuing a **TeleportAction**. When the item is used, it constructs and returns an action, which is executed like any other. The teleport logic is therefore unified with other teleport sources.

Design 1: Actor special-cases item	Design 2: Cube issues TeleportAction
------------------------------------	---

Pros: • Simple, no extra class	Pros: • DRY (reuses teleport action) • Items remain passive • Consistent action pathway
Cons: • Duplicates teleport logic • Tight coupling to actor	Cons: • Requires small helper/factory

Chosen design: *Design 2* – *Cube* uses *TeleportAction*, as it avoids duplication.

Why Design 2 was chosen

Design 2 was selected across all aspects of Requirement 1 because it provides a **cleaner, more extensible, and lower-coupled architecture** compared to Design 1. By moving the teleport trigger into the ground (*TeleportationCircle.onEnter*), we ensure that actors remain generic and are not burdened with environment-specific logic. Using a **registry-based destination resolution** avoids fragile hardcoded pairwise links and allows flexible connections across multiple maps. Encapsulating the move inside a dedicated *TeleportAction* keeps teleportation consistent with the FIT2099 action system, aligning with **SRP and OCP principles**.

Identifier-based tile linkage further reduces **connascence**, making teleport relationships explicit, testable, and easier to maintain as maps evolve. Finally, ensuring that the *TeleportCube* reuses the same *TeleportAction* avoids duplicated logic and keeps all teleportation behaviour centralised.

Overall, Design 2 was chosen because it is **scalable, modular, and easier to extend**, while Design 1 solutions would have introduced tight coupling, brittle dependencies, and violations of SOLID principles.